To avoid ambiguity, we need to be able to group instructions into *blocks*.

In its simplest definition, a block of instructions is a sequence of instructions that is executed sequentially from top to bottom.

As always, different programming languages have different ways to specify blocks: some of them use curly braces to mark beginning and ending of a block (f.i. C, Java etc…), some use explicitly the begin and end keywords (f.i. Pascal), and some others use indentation (Python).

Moreover, blocks can carry extra meaning in some programming languages, specifically if block-scope is used, then local variables defined inside a block can only be accessed inside that same block of code (including any nested block).

To simplify things, as mentioned, we won't bother with declaring variables, and we will instead assume function-scope, so variables can be accessed everywhere inside a function (see below). We will also assume lexical scope (aka static scope), so the life of variables ends when a function execution is completed, and there will be no support for *closures*.

Finally, blocks will be defined by indentation only: you can see the examples above are already indented, but the following example should clarify even further:

```
for i in {0..n-1} do
 k ← i * 2
```